



Universidad de Las Américas

**FACULTAD DE INGENIERÍA Y CIENCIAS APLICADAS
INGENIERÍA EN SOFTWARE**

ISWZ2202 – DISEÑO Y ARQUITECTURA DE SOFTWARE

PROYECTO FINAL DE ARQUITECTURA

CONTROLPRO ACADÉMICO

Arquitectura distribuida con microservicios, mensajería y API Gateway

Repositorio público:

github.com/Elnuget/ControlPro-Academico

ESTUDIANTE:

CARLOS ANGULO

DOCENTE:

ING. MÓNICA SÁNCHEZ

2026–2026

QUITO–ECUADOR

Resumen ejecutivo

ControlPro Académico evoluciona el blueprint inicial de seguimiento de proyectos universitarios hacia una solución distribuida, ejecutable y verificable. El ecosistema contiene exactamente tres aplicaciones conectadas: **API Gateway**, **Project Service** y **Alert Function**. La primera centraliza la API REST y Swagger; la segunda administra proyectos y avances con PostgreSQL y Redis; la tercera se comporta como una server function que consume eventos desde RabbitMQ y crea alertas sin bloquear la petición principal.

La solución aplica estilos de microservicios, arquitectura dirigida por eventos, API Gateway, funciones reactivas, cache-aside y Repository/Adapter. Su despliegue reproducible se realiza con Docker Compose y contiene pruebas unitarias, prueba de humo integral, logs estructurados y un flujo de integración continua en GitHub Actions.

Resultado verificable. La prueba integral obtuvo estado **ok**, registró un avance de 70 %, publicó el evento `progress.recorded` y confirmó la creación de una notificación `avance_registro` por la función asíncrona.

1. Antecedente y problema

El blueprint original identificó una necesidad frecuente: los avances, evidencias, rúbricas y observaciones de un proyecto académico se dispersan entre chats, correos, archivos locales y plataformas distintas. Esto reduce la trazabilidad para el estudiante y obliga al docente a reconstruir manualmente el estado del trabajo.

ControlPro centraliza el ciclo de seguimiento. El estudiante registra evidencia y porcentaje de avance; el docente consulta el historial; el sistema conserva datos transaccionales y genera alertas de manera desacoplada. El proyecto se mantiene deliberadamente pequeño para demostrar arquitectura, no para reemplazar un LMS institucional completo.

Actores: estudiante, docente y coordinador.

Caso principal: registrar un avance con evidencia y notificar su recepción.

Valor: trazabilidad, respuesta rápida, procesamiento resiliente y evolución independiente de componentes.

2. Objetivo y alcance

Objetivo general. Diseñar e implementar una solución distribuida de tres aplicaciones conectadas que cumpla los requisitos técnicos y documentales de la actividad de Diseño y Arquitectura de Software.

Alcance implementado:

- Crear y consultar proyectos académicos desde una única API pública.
- Registrar avances con porcentaje, evidencia, estudiante y fecha.
- Persistir proyectos, avances y notificaciones en PostgreSQL.
- Reducir lecturas repetidas mediante cache-aside en Redis con TTL de 60 segundos.
- Publicar `progress.recorded` en RabbitMQ y consumirlo desde una función.
- Documentar la API con OpenAPI 3.0 y Swagger UI.
- Desplegar aplicaciones y dependencias con Docker Compose.

Fuera del primer incremento: autenticación institucional real, carga binaria de evidencias, correo externo, aplicación móvil, calificación automática y alta disponibilidad productiva.

3. Trazabilidad de la consigna

Requisito	Evidencia en ControlPro
Tres aplicaciones conectadas	<code>api-gateway</code> , <code>project-service</code> y <code>alert-function</code> ; comunicación REST y AMQP.
Funcionalidad simple	Crear/listar proyectos, registrar avances y consultar alertas.
Capa de datos en Docker	PostgreSQL 16 para persistencia y Redis 7 para caché.
Server Functions o Lambdas	<code>alert-function</code> reacciona a cada evento de avance.
Microservicios y gestor de colas	RabbitMQ con exchange topic y cola durable <code>controlpro.alerts</code> .
API Gateway RESTful	Puerto 3000 como única entrada pública; servicios internos no se publican.
Swagger	<code>/docs</code> y contrato descargable en <code>/openapi.json</code> .
Diagramas de arquitectura	Diagramas C4 de contexto, contenedores y componentes.

Infraestructura y despliegue	Docker Compose, Dockerfiles, volúmenes, health checks y reinicio.
Análisis de arquitectura	Caché, balanceo, indexación, redundancia, disponibilidad, concurrencia, latencia, costo, performance y escalabilidad.
Mantenibilidad, logs y CI/CD	Tests, validación, logs JSON, <code>x-request-id</code> y GitHub Actions.

4. Las tres aplicaciones del ecosistema

Aplicación	Tecnologías	Responsabilidad
API Gateway	Node.js, Express, Swagger, Helmet	Entrada REST única; contrato OpenAPI; correlación de peticiones; timeout y manejo uniforme de indisponibilidad.
Project Service	Node.js, Express, PostgreSQL, Redis, RabbitMQ	Dueño del dominio; valida, persiste, consulta, usa caché y publica eventos confirmados.
Alert Function	Node.js, AMQP, PostgreSQL	Consumidor reactivo; transforma eventos en alertas e impide duplicados con <code>event_id</code> .

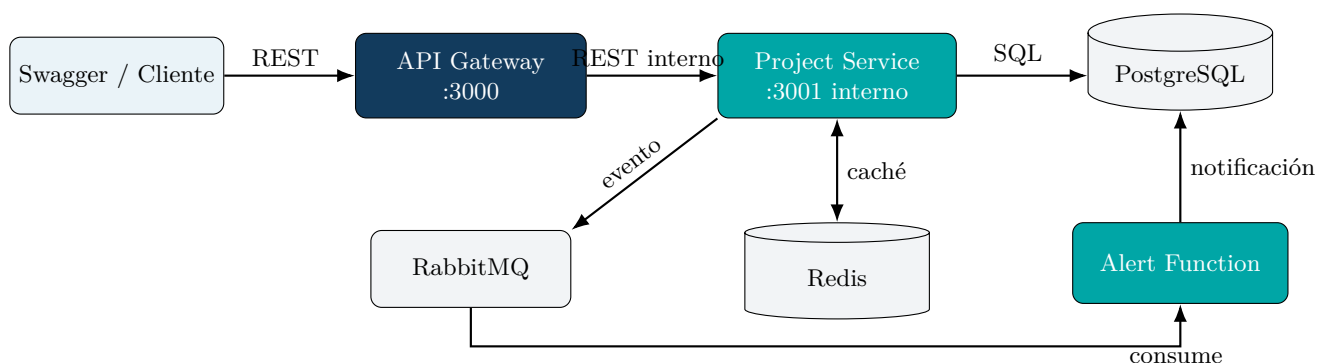
5. Estilos y patrones arquitectónicos

- **API Gateway:** oculta topología interna, concentra documentación y define una frontera pública estable.
- **Microservicios ligeros:** cada proceso tiene responsabilidad, dependencias y despliegue independiente.
- **Event-Driven Architecture:** el registro de avance publica un hecho inmutable para procesamiento posterior.
- **Server Function:** la función de alertas solo se activa ante mensajes y puede escalar con consumidores adicionales.
- **Repository/Adapter:** HTTP, SQL, caché y mensajería se mantienen como detalles alrededor del caso de uso.
- **Cache-aside:** se consulta Redis antes de PostgreSQL y se invalida la lista cuando cambia un proyecto.
- **Idempotent Consumer:** `event_id` UNIQUE evita alertas duplicadas ante reentregas.

6. Diagrama C4 – Contexto



7. Diagrama C4 – Contenedores



8. Flujo funcional principal

1. El cliente invoca `POST /api/projects/{id}/progress` en el Gateway.
2. El Gateway asigna o propaga `x-request-id` y reenvía la petición con timeout de cinco segundos.

3. Project Service valida el porcentaje y la evidencia, bloquea el proyecto y abre una transacción.
4. PostgreSQL inserta el historial y actualiza porcentaje y estado; la transacción confirma ambos cambios juntos.
5. El servicio publica `progress.recorded` como mensaje durable y espera confirmación del broker.
6. El cliente recibe HTTP 201 sin esperar el procesamiento de la alerta.
7. Alert Function consume el mensaje, construye la notificación y la inserta de forma idempotente.
8. La alerta puede comprobarse con `GET /api/notifications`.

9. API Gateway y documentación Swagger

Swagger UI está disponible en <http://localhost:3000/docs>; el contrato OpenAPI 3.0 también se expone como JSON. El Project Service permanece dentro de la red de Docker.

Método	Ruta pública	Finalidad
GET	<code>/health</code>	Estado del servicio y dependencias.
GET	<code>/api/projects</code>	Lista proyectos; indica HIT/MISS de caché.
POST	<code>/api/projects</code>	Crea un proyecto académico.
GET	<code>/api/projects/{id}</code>	Detalle e historial de avances.
POST	<code>/api/projects/{id}/progress</code>	Persiste avance y publica evento.
GET	<code>/api/notifications</code>	Evidencia alertas creadas por la función.

10. Datos, caché y mensajería

PostgreSQL. Las tablas `projects`, `progress_updates` y `notifications` mantienen claves UUID, restricciones de rango, referencias y fechas auditables. El avance se actualiza dentro de una transacción con bloqueo `FOR UPDATE`.

Redis. La clave `projects:active` conserva por 60 segundos la consulta más frecuente. La caché se invalida al crear proyectos o registrar avances, evitando mostrar porcentajes obsoletos.

RabbitMQ. Un exchange topic durable `controlpro.events` enruta `progress.recorded` hacia `controlpro.alerts`. Los mensajes son persistentes, el consumidor limita su prefetch y confirma solo después de insertar la alerta.

11. Análisis obligatorio de la arquitectura

Caché

Redis reduce lecturas repetidas del tablero. El TTL limita obsolescencia y la invalidación explícita mantiene coherencia después de escrituras. No se cachean los avances individuales porque su trazabilidad requiere datos actuales.

Balanceo

Las aplicaciones no guardan sesión local, por lo que se pueden ejecutar varias réplicas detrás de un balanceador L7 o un Kubernetes Service. El Gateway es la única entrada externa; Project Service y los consumidores se balancean dentro de la red.

Indexación

Se crean índices por `deadline`, `status`, `project_id + created_at` y notificaciones por proyecto/fecha. Estos índices mejoran tableros e historiales; su costo es un pequeño incremento en escritura y almacenamiento.

Redundancia y disponibilidad

Docker Compose demuestra recuperación con `restart: unless-stopped`. En producción se proponen dos réplicas de aplicaciones, PostgreSQL administrado con réplica, Redis administrado y RabbitMQ en clúster. La cola conserva trabajo mientras la función se recupera.

Concurrencia

`SELECT ... FOR UPDATE` serializa actualizaciones simultáneas del mismo proyecto. La restricción de 0 a 100 se aplica tanto en la validación como en la base. El consumidor idempotente soporta reentrega sin duplicar resultados.

Latencia

La ruta crítica contiene Gateway, Project Service y PostgreSQL. El procesamiento secundario sale de esa ruta mediante RabbitMQ. El timeout del Gateway evita esperas indefinidas y Redis acelera lecturas repetidas.

Costo y proyección

El entorno académico solo necesita Docker local. Para una primera producción pequeña se estima un VPS con aplicaciones y servicios administrados básicos; al crecer, el mayor costo estará en PostgreSQL de alta disponibilidad, almacenamiento y observabilidad. La función puede migrar a Lambda/Azure Functions para pago por ejecución.

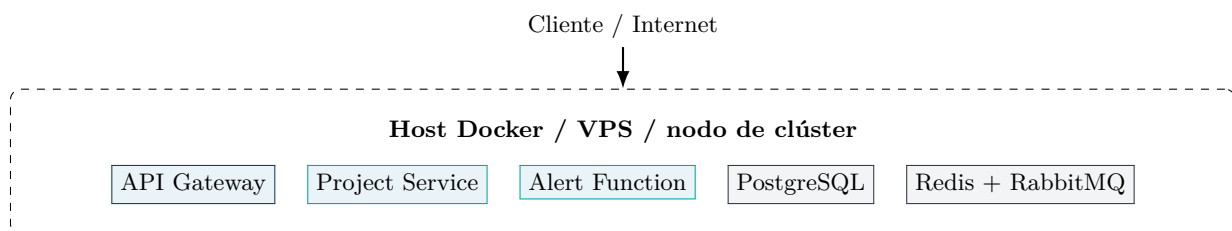
Performance y escalabilidad

Gateway y Project Service escalan horizontalmente. RabbitMQ absorbe ráfagas y permite aumentar consumidores de Alert Function. Las métricas decisivas son solicitudes por segundo, p95 de latencia, hit ratio de Redis, conexiones SQL, profundidad de cola y tasa de errores.

12. Seguridad, mantenibilidad y observabilidad

- Helmet agrega cabeceras seguras y el cuerpo JSON se limita a 256 KB.
- La frontera pública contiene validación y errores sin detalles internos.
- Para producción se proyecta JWT/SSO y RBAC para estudiante, docente y coordinador.
- Las credenciales se parametrizan; `.env` está ignorado y `.env.example` no contiene secretos reales.
- Los servicios emiten logs JSON y el Gateway propaga `x-request-id`.
- Health check verifica PostgreSQL y Redis; RabbitMQ expone consola de operación.
- GitHub Actions ejecuta instalación reproducible, cuatro pruebas, análisis sintáctico, validación de Compose y construcción de imágenes.

13. Infraestructura y despliegue



```
git clone https://github.com/Elnuget/ControlPro-Academico.git
cd ControlPro-Academico
docker compose up --build -d
powershell -ExecutionPolicy Bypass -File scripts\smoke-test.ps1
```

14. Integración y despliegue continuo

El flujo CI de GitHub Actions se activa en cada push a `main` y en pull requests. Instala Node 20, ejecuta las pruebas unitarias, revisa sintaxis, valida la configuración y construye las tres imágenes. Para despliegue continuo se propone añadir un registro de contenedores y una etapa protegida que actualice un VPS o clúster solo cuando CI sea exitoso.

Proceso recomendado: commit pequeño → push → CI → imagen versionada → aprobación → despliegue → smoke test → monitoreo y rollback si falla.

15. Evidencia de funcionamiento

La verificación local se ejecutó el 9 de julio de 2026 con Docker Desktop. La evidencia obtenida fue:

Comprobación	Resultado
Pruebas unitarias	4 aprobadas, 0 fallidas.
Análisis sintáctico	Tres aplicaciones válidas con <code>node -check</code> .
Docker Compose	PostgreSQL y Redis saludables; seis contenedores activos.
Gateway	<code>/health</code> respondió <code>status: ok</code> .
Persistencia	Proyecto semilla consultado desde PostgreSQL.
Mensajería	Evento <code>progress.recorded</code> publicado y confirmado.
Server Function	Notificación <code>avance_registro</code> creada y consultada por API.
Seguridad de dependencias	Auditoría NPM: 0 vulnerabilidades reportadas.

Identificadores de la ejecución de evidencia. Proyecto: 11111111-1111-4111-8111-111111111111. Evento: 35507170-3d8e-4527-9caf-0a445bad5c0e.

16. Guion de demostración para clase

1. Mostrar el diagrama de contenedores y explicar por qué existen tres aplicaciones.
2. Ejecutar `docker compose ps` para evidenciar infraestructura activa.
3. Abrir Swagger en <http://localhost:3000/docs>.
4. Consultar `GET /api/projects` y copiar el identificador del proyecto.
5. Ejecutar `POST /api/projects/{id}/progress` con 70% y una evidencia.
6. Señalar que HTTP 201 confirma persistencia y publicación del evento.
7. Consultar `GET /api/notifications` y mostrar la alerta creada por la función.
8. Cerrar relacionando el flujo con caché, concurrencia, disponibilidad y escalabilidad.

Conclusiones

ControlPro Académico cumple la consigna con una solución ejecutable, comprensible y coherente con el blueprint original. Las tres aplicaciones separan entrada, dominio y procesamiento asíncrono; PostgreSQL protege consistencia, Redis mejora lectura y RabbitMQ desacopla alertas. La solución no simula únicamente diagramas: dispone de contrato Swagger, contenedores reproducibles, pruebas y una ejecución integral verificada.

La arquitectura puede evolucionar sin reemplazar su núcleo: nuevos consumidores pueden procesar correos o reportes; las aplicaciones pueden replicarse; y las dependencias pueden migrar a servicios administrados. Este equilibrio evita sobrediseño académico y, al mismo tiempo, evidencia decisiones profesionales de mantenibilidad, operación y escalabilidad.

Repositorio y artefactos

- Código fuente y documentación: <https://github.com/Elnuget/ControlPro-Academico>
- Blueprint base: `ControlPro_Academico_Blueprint_v1.drawio`
- Swagger local: <http://localhost:3000/docs>
- Diagramas editables en Mermaid: `docs/architecture.md`